



CO5-AT2-Application Based Problem Solving

[← Back](#)

QUESTIONS

Q1Question 3 — E-Commerce
Order & Inventory System
100 marks

1/1 answered

Q1 Question 3 — E-Commerce Order & Inventory System

100 marks

Design an Item base class and subclasses Electronics and Grocery with overridden calculateDiscount() methods (different discount logic each). Use constructor overloading for items with/without a discount code. Overload a placeOrder() method to accept a single item or a list of items. Define OutOfStockException and InvalidQuantityException. Implement inventory reduction as a method accessed by multiple customer threads simultaneously ordering the same product with limited stock. Evaluate: Analyze a race condition scenario where two threads both check stock > 0 before decrementing it, causing overselling. Propose and justify a fix (synchronized block vs. AtomicInteger vs. ReentrantLock), evaluating trade-offs in performance and code complexity for a high-traffic checkout system.

Your Code

```
1 import java.util.*;
2 import java.util.concurrent.atomic.AtomicInteger;
3 import java.util.concurrent.locks.ReentrantLock;
4
5 // Custom Exception 1
6 class OutOfStockException extends Exception {
7     public OutOfStockException(String message) {
```

Input

stdin input

```
8         super(message);
9     }
10 }
11
12 // Custom Exception 2
13 class InvalidQuantityException extends Exception {
14     public InvalidQuantityException(String message)
15         super(message);
16     }
17 }
18
19 // Base class
20 abstract class Item {
21     protected String name;
22     protected double price;
23     protected int stock;
24     protected String discountCode;
25
26     // Constructor without discount code
27     public Item(String name, double price, int stock)
28         this(name, price, stock, null);
29     }
30
31     // Constructor with discount code
32     public Item(String name, double price, int stock, String discountCode)
33         this.name = name;
34         this.price = price;
35         this.stock = stock;
36         this.discountCode = discountCode;
37     }
38
39     // Overridden by subclasses
40     public abstract double calculateDiscount();
```

Output

Compilation Error:
C:\Windows\TEMP\sandbo
x_192324238RB_6a8ea29b
86d640.48386781\OutOfS
tockException.java:138
: error: class

```
41
42     public synchronized void reduceStock(int quantity, double price)
43         throws OutOfStockException, InvalidQuantityException {
44
45         if (quantity <= 0)
46             throw new InvalidQuantityException("Quantity cannot be less than 0");
47
48         if (quantity > stock)
49             throw new OutOfStockException("Out of stock for " + name + ".");
50
51         stock -= quantity;
52
53         System.out.println(Thread.currentThread().getName() + " " +
54             + " purchased " + quantity + " " + price + " " +
55             + " | Remaining stock: " + stock);
56     }
57
58
59     public double finalPrice() {
60         return price - calculateDiscount();
61     }
62
63     public String getName() {
64         return name;
65     }
66 }
67
68 // Electronics subclass
69 class Electronics extends Item {
70
71     public Electronics(String name, double price, int stock, double discount) {
72         super(name, price, stock);
73     }
```

```
74
75     public Electronics(String name, double price, int stock, String discountCode) {
76         super(name, price, stock, discountCode);
77     }
78
79     @Override
80     public double calculateDiscount() {
81         // Electronics: 10% discount if discount code is 'E'
82         if (discountCode != null && discountCode.equals("E"))
83             return price * 0.10;
84
85         return 0;
86     }
87 }
88
89 // Grocery subclass
90 class Grocery extends Item {
91
92     public Grocery(String name, double price, int stock, String discountCode) {
93         super(name, price, stock);
94     }
95
96     public Grocery(String name, double price, int stock, String discountCode) {
97         super(name, price, stock, discountCode);
98     }
99
100    @Override
101    public double calculateDiscount() {
102        // Grocery: 5% discount if discount code is 'G'
103        if (discountCode != null && discountCode.equals("G"))
104            return price * 0.05;
105
106        return 0;
107    }
108 }
```

```
107     }
108 }
109
110 // Order class
111 class Order {
112
113     // Overloaded method: single item
114     public void placeOrder(Item item, int quantity)
115     {
116         try {
117             item.reduceStock(quantity);
118
119             System.out.println("Order placed for: " +
120                 System.out.println("Final Price: ₹" +
121                     (item.finalPrice() * quantity))
122                 System.out.println());
123         } catch (Exception e) {
124             System.out.println(e.getMessage());
125         }
126     }
127
128     // Overloaded method: list of items
129     public void placeOrder(List<Item> items) {
130         System.out.println("Placing multiple-item c
131
132         for (Item item : items) {
133             placeOrder(item, 1);
134         }
135     }
136
137 // Main class
138 public class InventorySystem {
139
```

```
140     public static void main(String[] args) {
141
142         Electronics laptop =
143             new Electronics("Laptop", 60000, 3,
144
145         Grocery rice =
146             new Grocery("Rice", 1000, 5, "GROC
147
148         Order order = new Order();
149
150         // Single item order
151         order.placeOrder(laptop, 1);
152
153         // Multiple item order
154         List<Item> items = new ArrayList<>();
155         items.add(rice);
156         items.add(laptop);
157
158         order.placeOrder(items);
159
160         // Demonstrating multithreading
161         System.out.println("\n--- Concurrent Orderi
162
163         Electronics phone =
164             new Electronics("Smartphone", 30000
165
166         Runnable task = () -> {
167             order.placeOrder(phone, 1);
168         };
169
170         Thread t1 = new Thread(task, "Customer-1");
171         Thread t2 = new Thread(task, "Customer-2");
172         Thread t3 = new Thread(task, "Customer-3");
```

```
173
174     t1.start();
175     t2.start();
176     t3.start();
177
178     try {
179         t1.join();
180         t2.join();
181         t3.join();
182     } catch (InterruptedException e) {
183         Thread.currentThread().interrupt();
184     }
185
186     System.out.println("Final remaining stock:");
187 }
188 }
```


✔ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.